

Introduction to Quantum Machine Learning

Ibrahim Shehzad
Quantum Algorithm Engineer
IBM Quantum

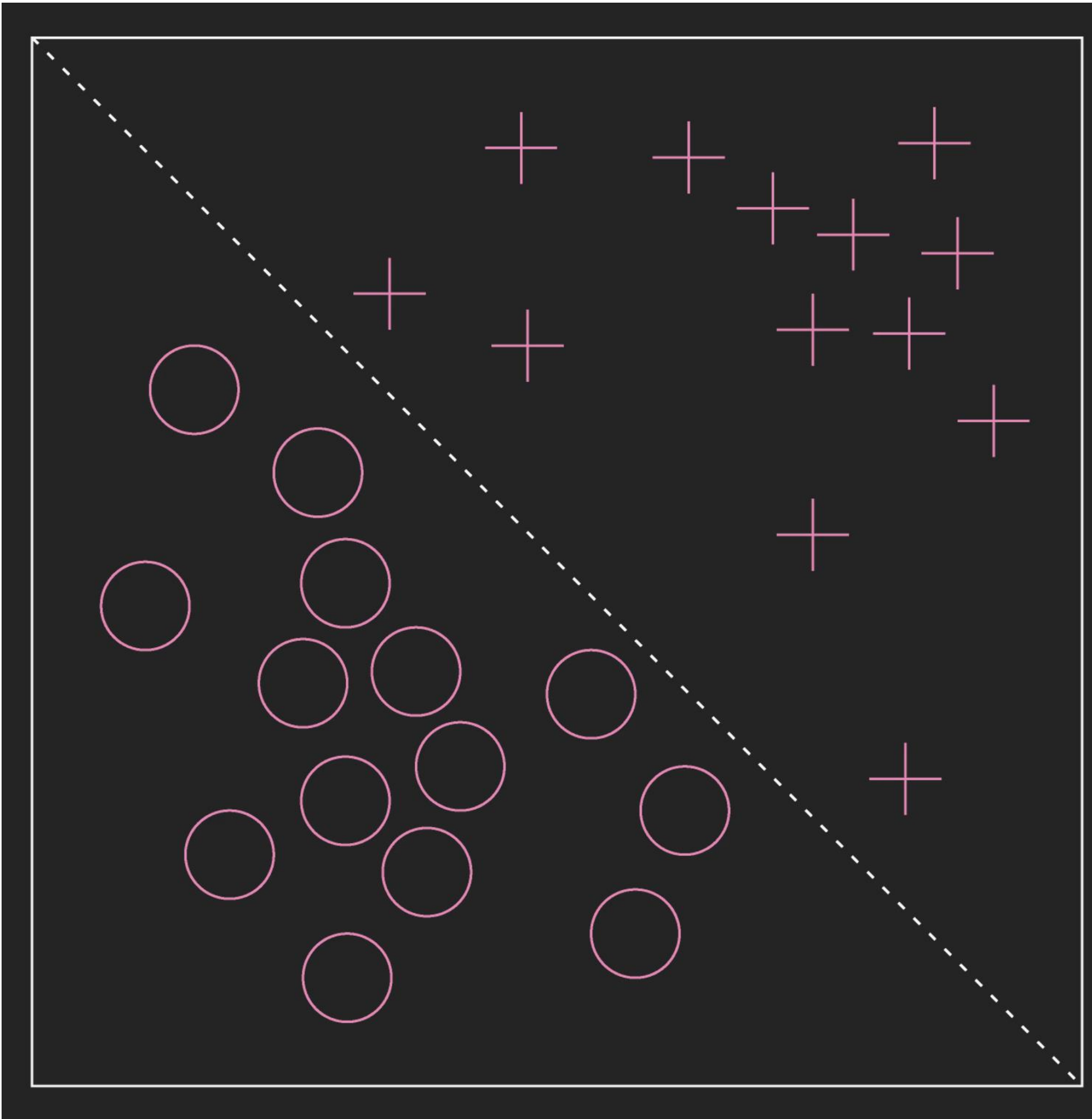
Overview

- **Introduction**
 - What is machine learning?
 - Motivation: Why use Quantum Machine Learning?
- **Overview machine learning methods**
 - Types of Classical Machine Learning
 - Going from Classical to Quantum
 - Support vector machines
 - Kernel Methods
 - Other types of Quantum Machine Learning
 - Quantum Neural Networks
- **Data encoding methods**
 - Lossless vs Lossy encoding
 - Basis Encoding
 - Amplitude Encoding
 - Angle Encoding
 - Phase Encoding
 - Dense Angle Encoding

What is machine learning?

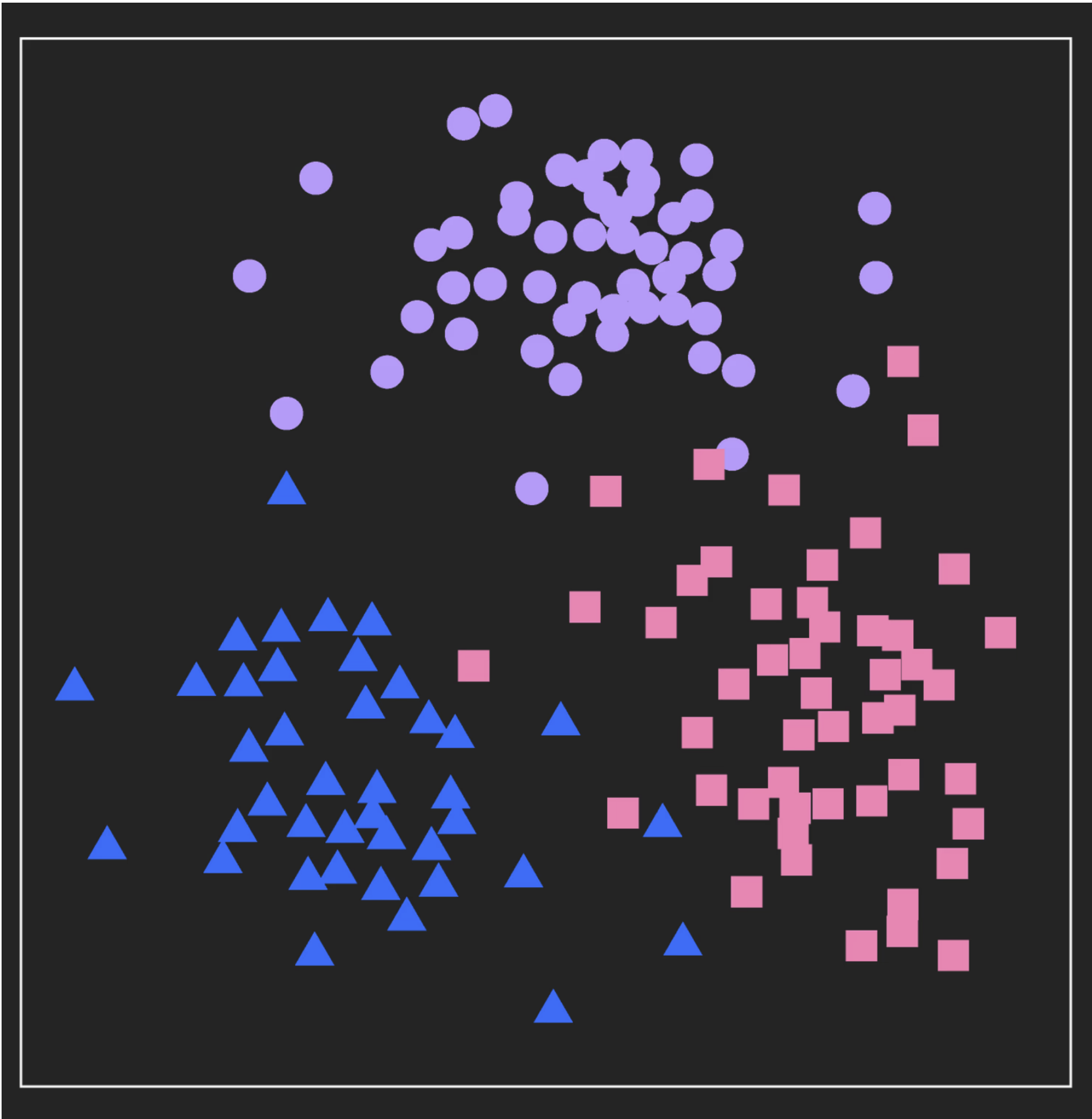
- A subfield of Artificial Intelligence.
- Statistical Algorithms.
- Apply algorithms to (huge) data sets.
- Finds **structure in data**.
- Has a close connection to data compression.
 - AI models are a lot smaller than the total data processed.
 - Example: classifying data in groups.
- Often involves **mapping data into higher-dimensional spaces**.

A specific type of machine learning: Classification.



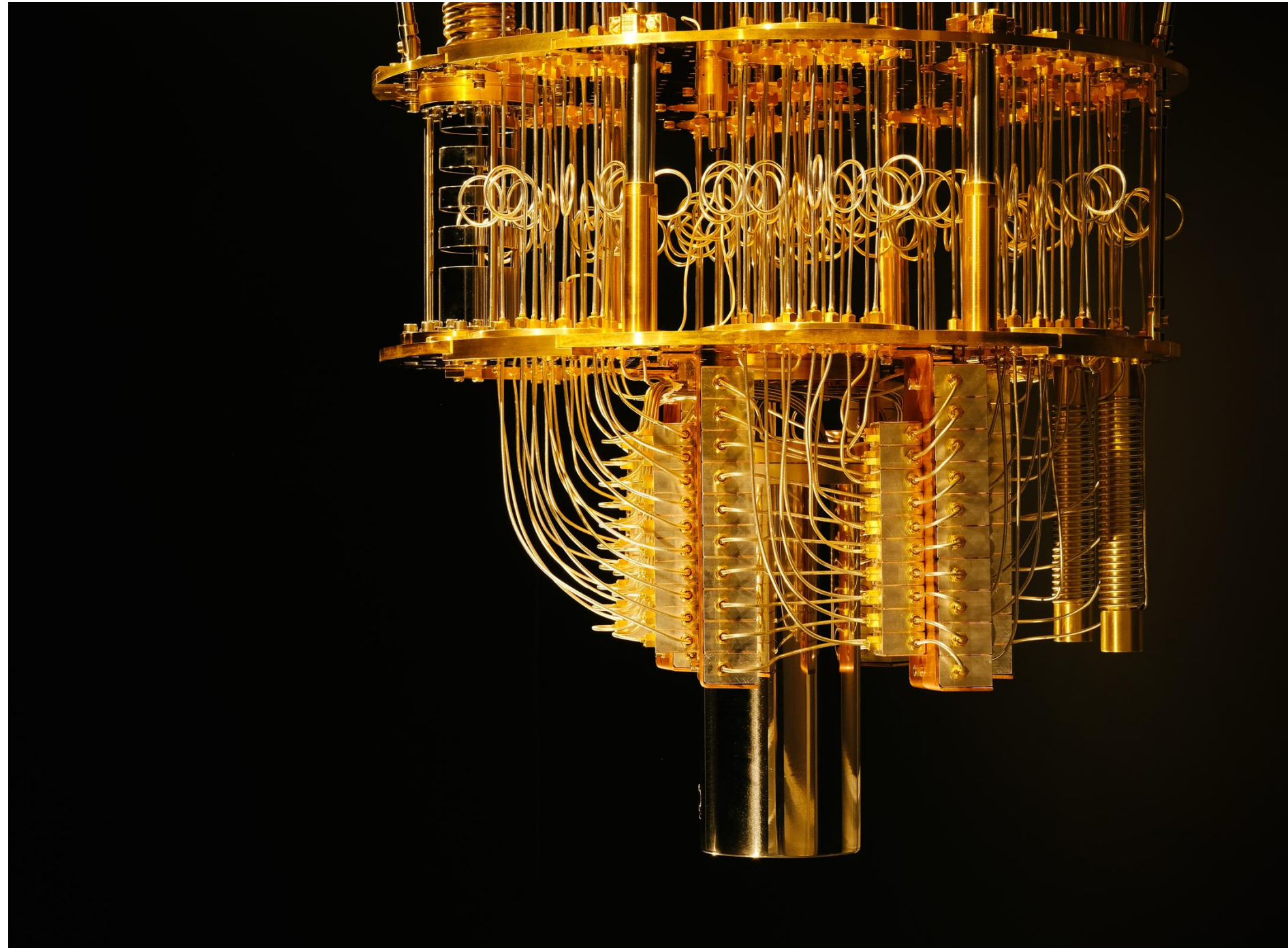
- [Binary classification](#) (using 2D data):
 - We know there are 2 types of data points.
 - Each datapoint has 2 values represented by X and Y.
 - Classifying data by finding a line (or plane) to separate them.
- Example for binary classifications:
 - Captchas: “Click all images with cars.”
 - High energy physics: Distinguish between particle and anomalies.

A specific type of machine learning: Classification.



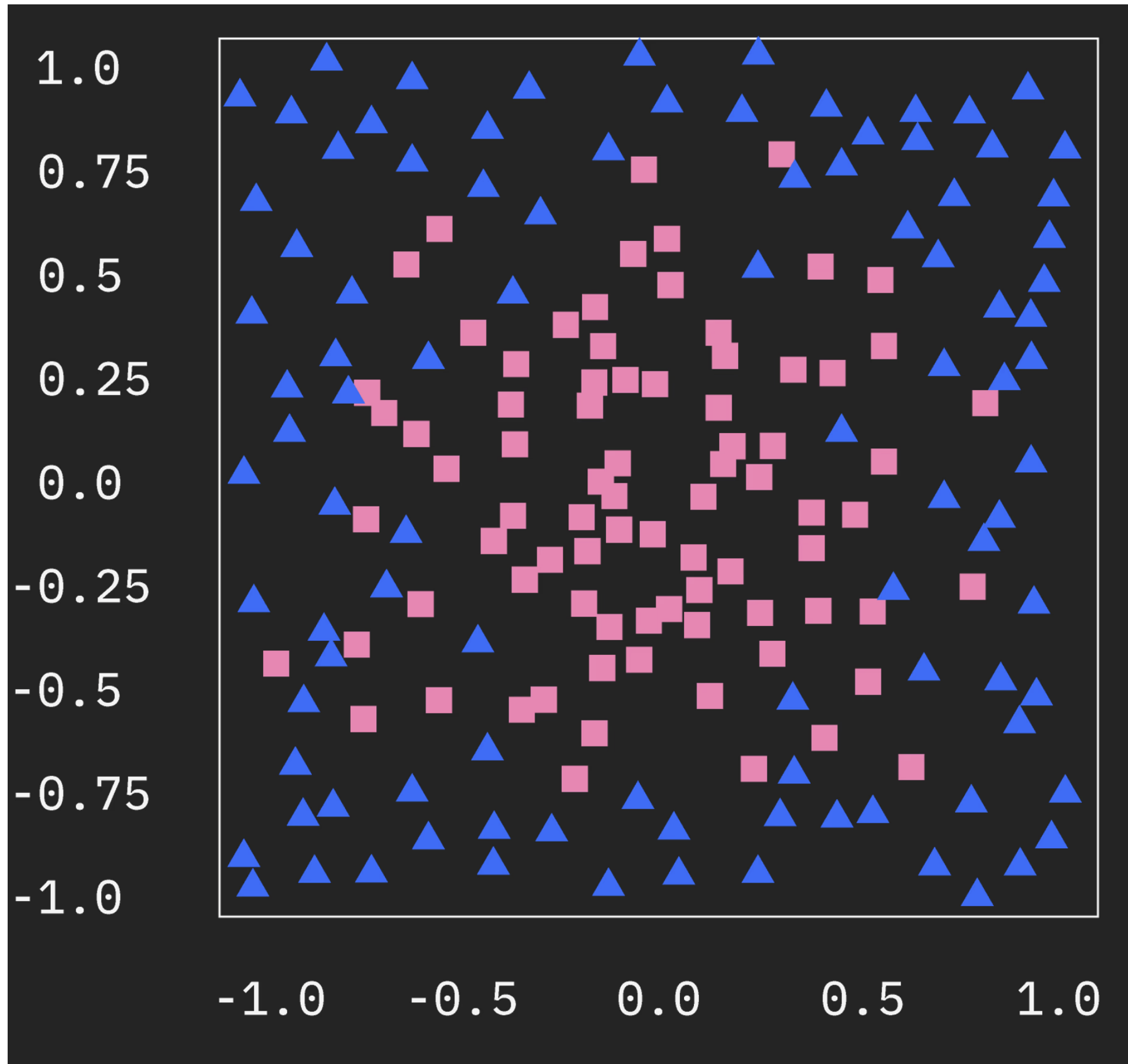
- General classification (using 2D data):
 - Classifying data into several groups
 - Data often has overlap
 - Solution: map to higher dimensions
- Example for general classifications:
 - Movie recommendations: Recommending similar movie.
 - Image recognition: What is on the image?

Motivation: Why use Quantum?



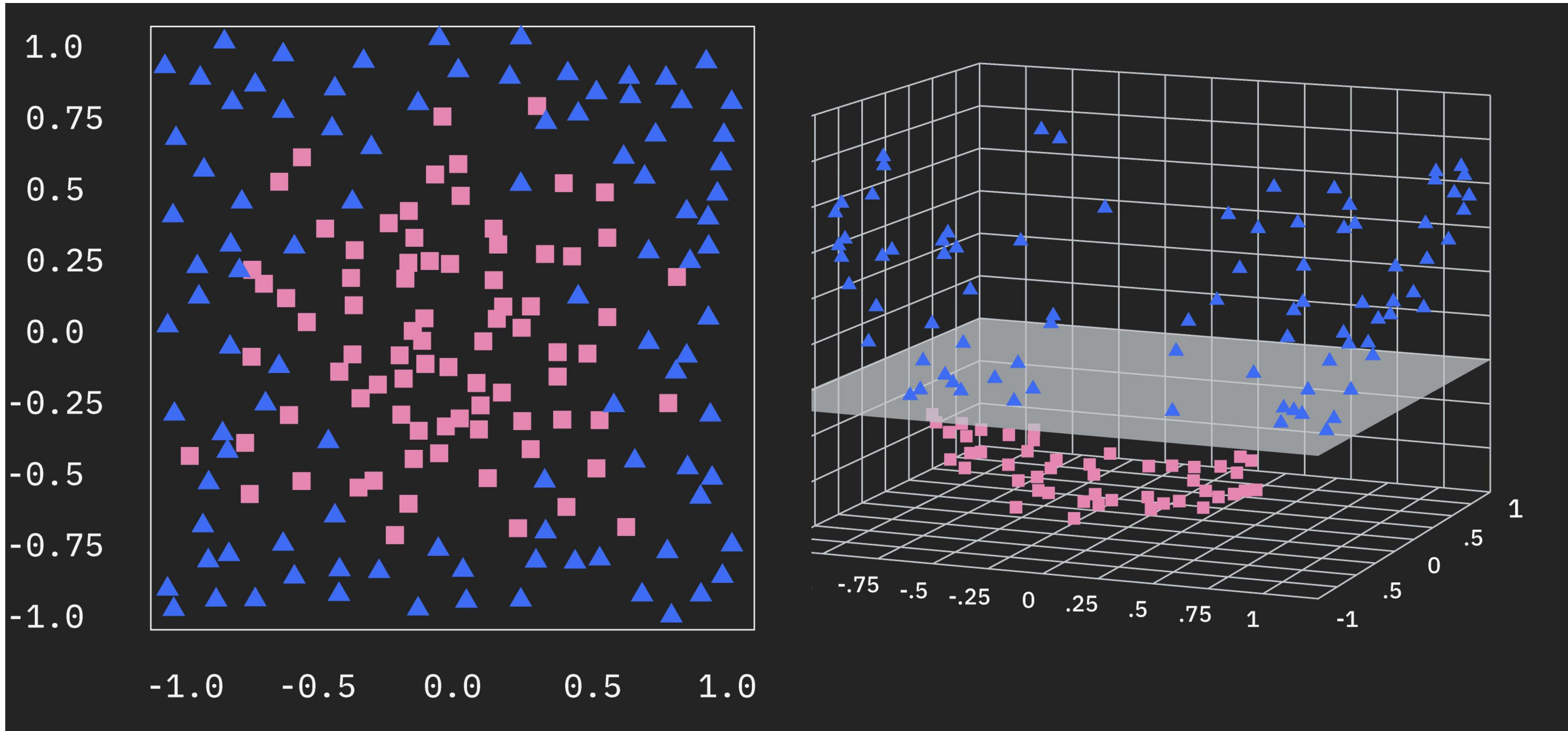
- Different types of computing are good for different data.
- Some physical data can be better represented in quantum states.
- Often involves mapping data into higher-dimensional spaces.
- Quantum states allow us to represent data in high dimensions!

Why are higher dimensions important?

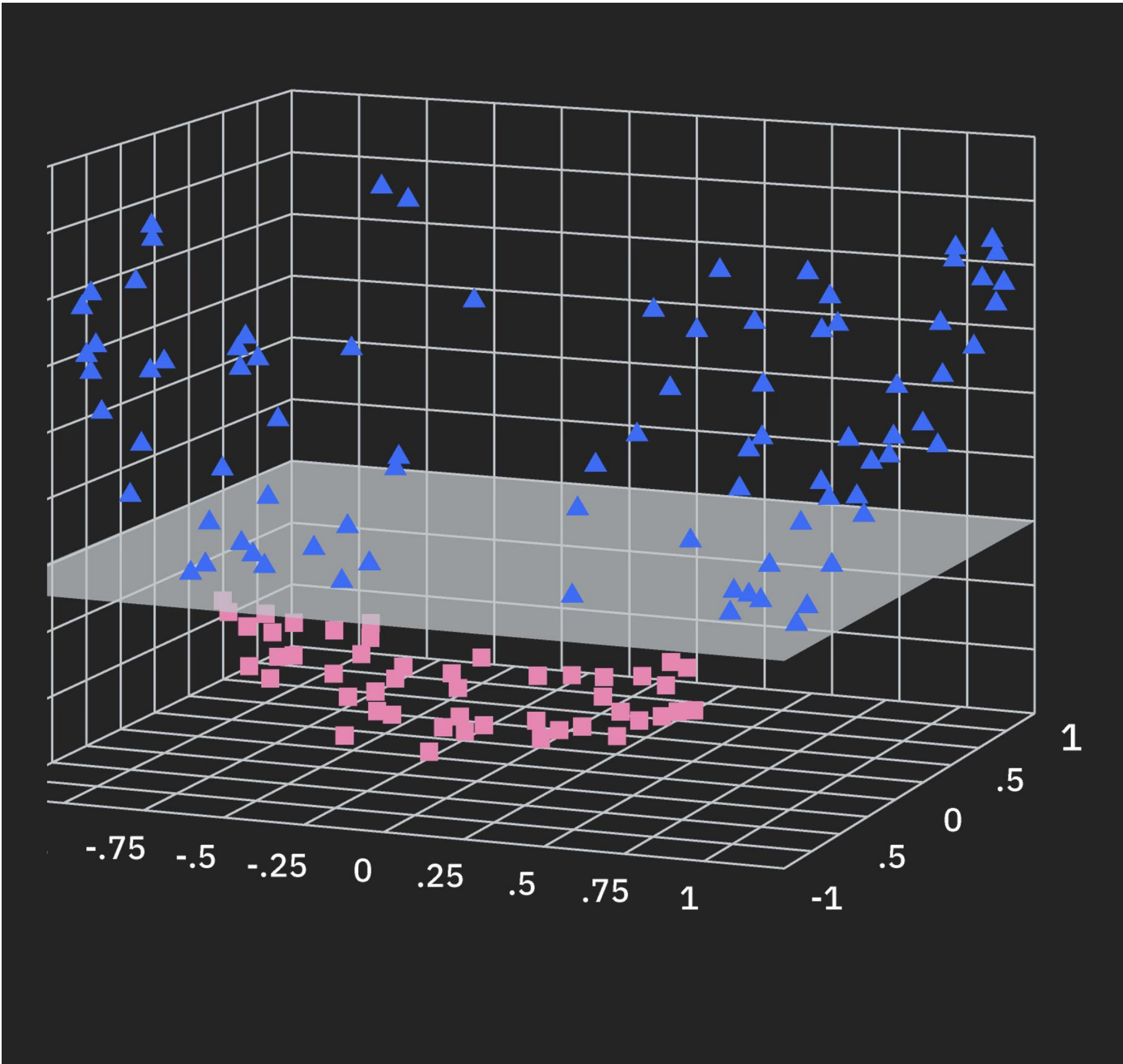


- Data may be hard to classify in low dimensions.
- Different groups of data are mixed together.
- No clear mapping can be found to distinguish in low dimensions.

Lets go to higher dimensions!



Why are higher dimensions important?



- In higher dimensions data might be easier to separate.
- Finding a hyperplane to separate data
- However, higher dimensions is not magic:
 - Just by going to higher dimensions does not automatically makes it separable
 - We need a good mapping.
 - And this may not work if data is inconsistent.

Quantum computing naturally has high dimensions.

$$2^4 = 16$$

$$(1, 0, 0, 0), (0, 1, 0, 0), \dots (1, 1, 1, 1)$$

$$a(1, 0, 0, 0) + b(0, 1, 0, 0) + c(0, 0, 1, 0) + \dots$$

$$a, b, c \dots \in \mathbb{C}, |(a, b, c \dots)|^2 = 1$$

- In classical data only fixed bits are possible: So we only have pure states.
- In quantum data we have superposition, allowing us to have a combination of states.
- This allows for exponentially many parameters.
- This also allows us to encode data in different ways, potentially needing a lot less qubits than bits.
- Additionally, [some operations can be done efficiently](#).

Overview

- ✓ • **Introduction**
 - What is machine learning?
 - Motivation: Why use Quantum Machine Learning?
- **Overview machine learning methods**
 - Types of Classical Machine Learning
 - Going from Classical to Quantum
 - Support vector machines
 - Kernel Methods
 - Other types of Quantum Machine Learning
 - Quantum Neural Networks
- **Data encoding methods**
 - Lossless vs Lossy encoding
 - Basis Encoding
 - Amplitude Encoding
 - Angle Encoding
 - Phase Encoding
 - Dense Angle Encoding

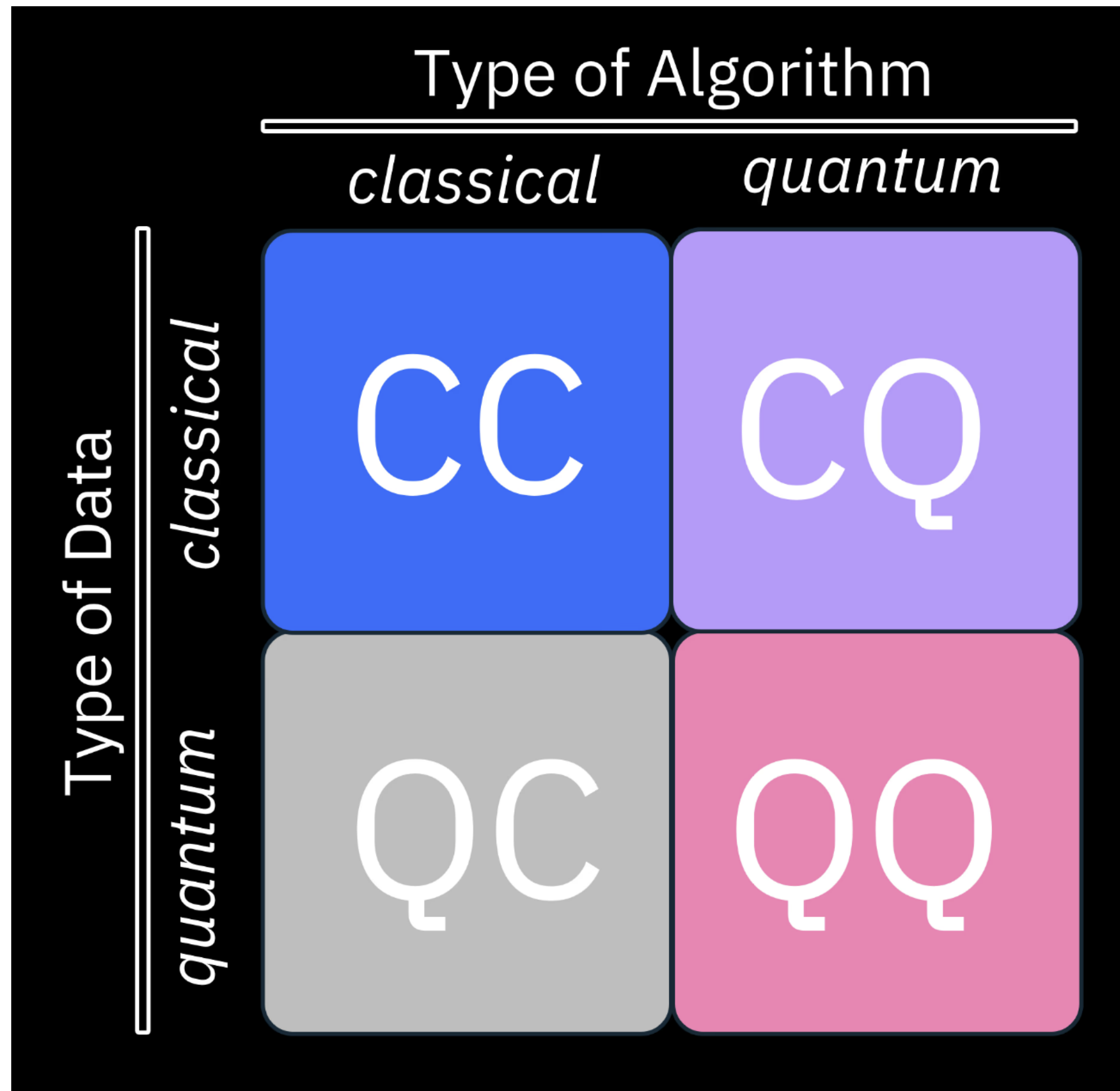
Types of classical machine Learning

1. Supervised machine learning.
2. Unsupervised machine learning.
3. Reinforcement learning.
4. Self-supervised machine learning.
5. Semi-supervised learning.

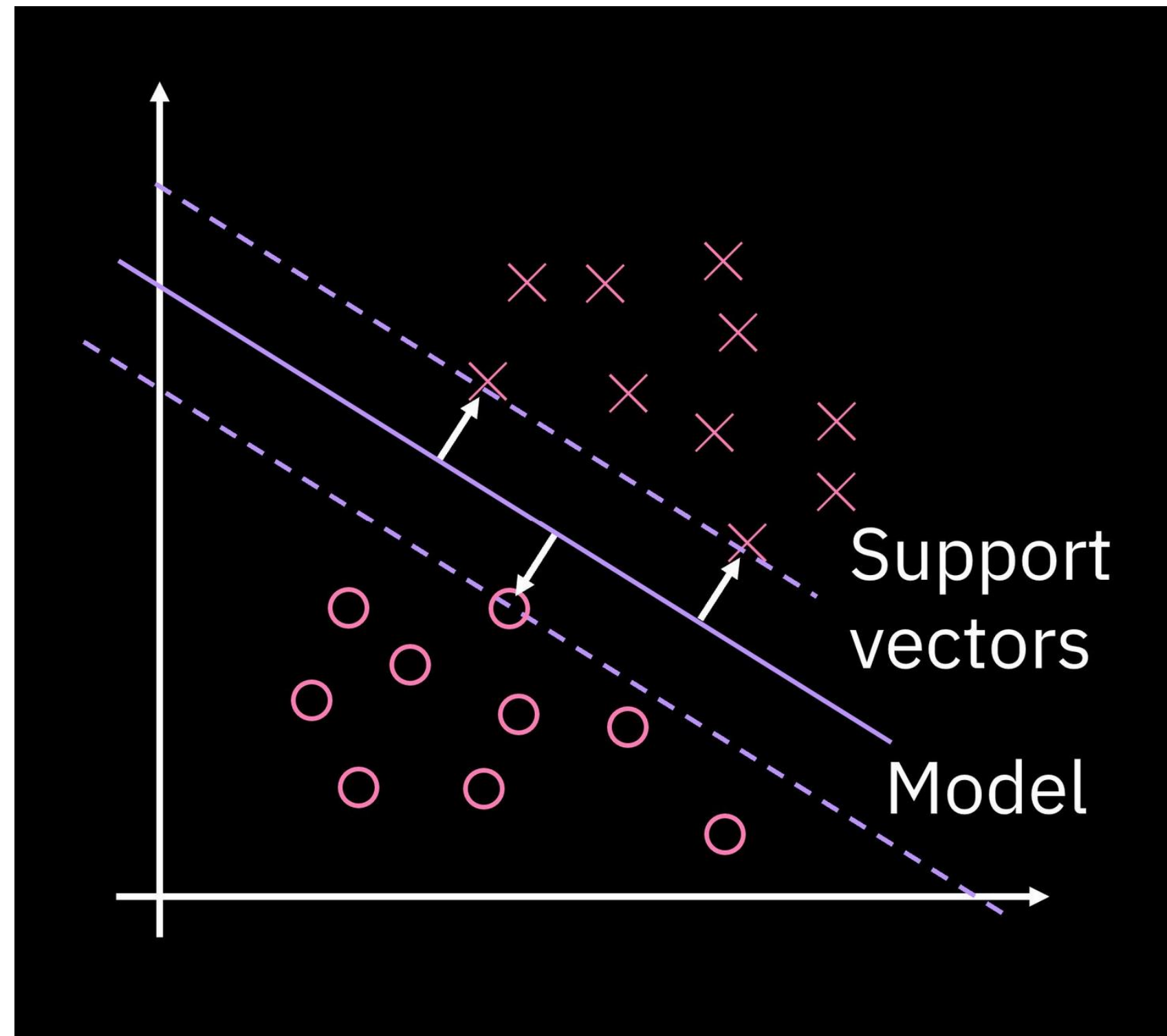
In **supervised learning**, the **data** that is used to train the model is **labeled**. The goal of these algorithms is to learn the relationship between data and their corresponding labels or outputs and to generalize this to unseen data. Common tasks in this class are **classification** and **regression**.

In contrast, **unsupervised learning** uses **unlabeled data** to train the machine learning model. The goal of such algorithms is to discover hidden patterns and structure in data. Some algorithms in this class are **clustering** and **dimensionality reduction** algorithms.

Going from Classical to Quantum Machine Learning



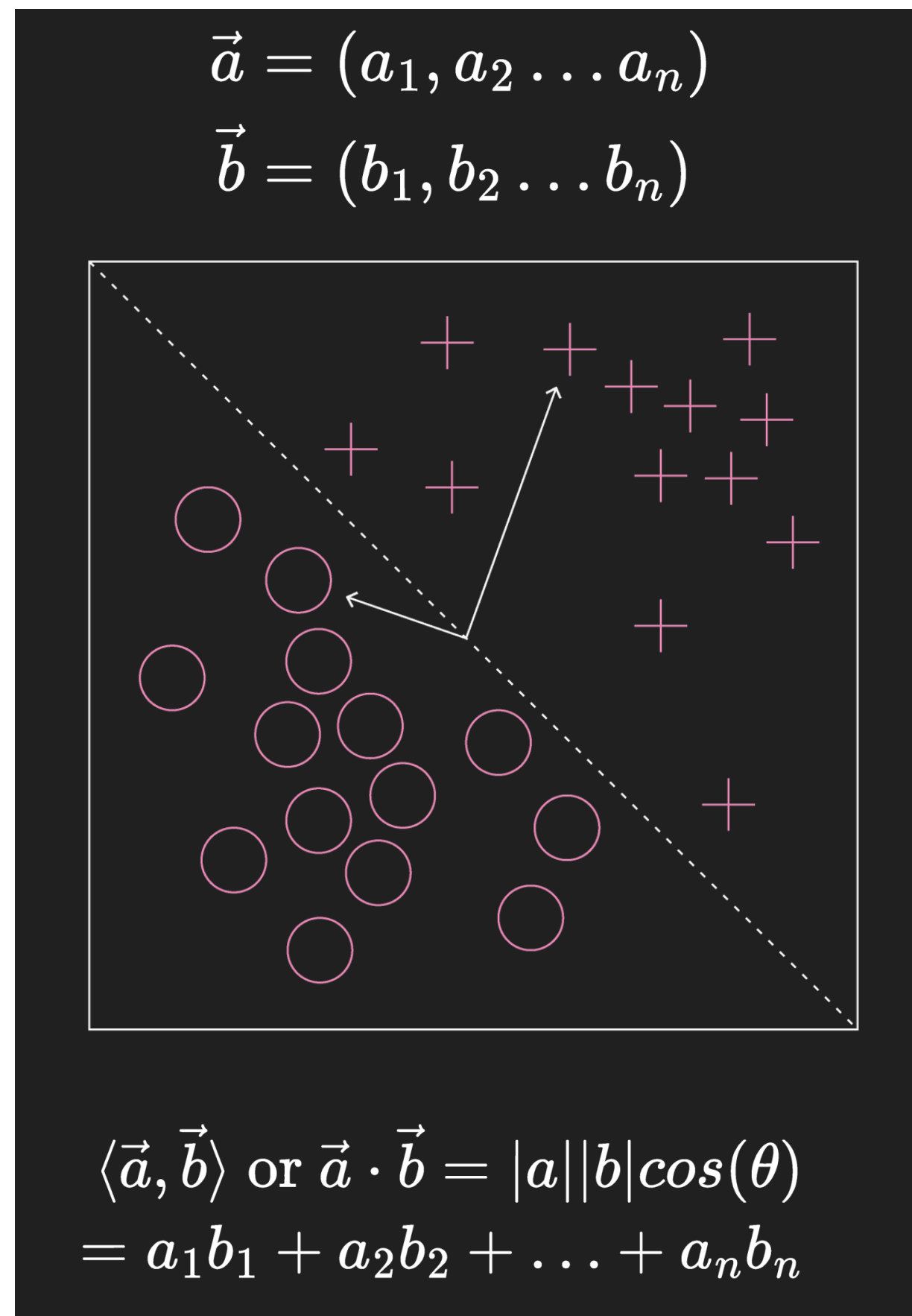
1. Support vector machines
2. Using the dot-product
3. Kernel Methods
4. Quantum Kernels



The **arrows** show the **support vectors**.

- Plan is to find a boundary to separate different groups as clearly as possible.
- We want to maximize the minimal distance to points!
- We often want to map the input points to a higher dimension to make it easier to find a clear boundary.
- We want to **learn** the optimal **parameters of the separating hyperplane**.

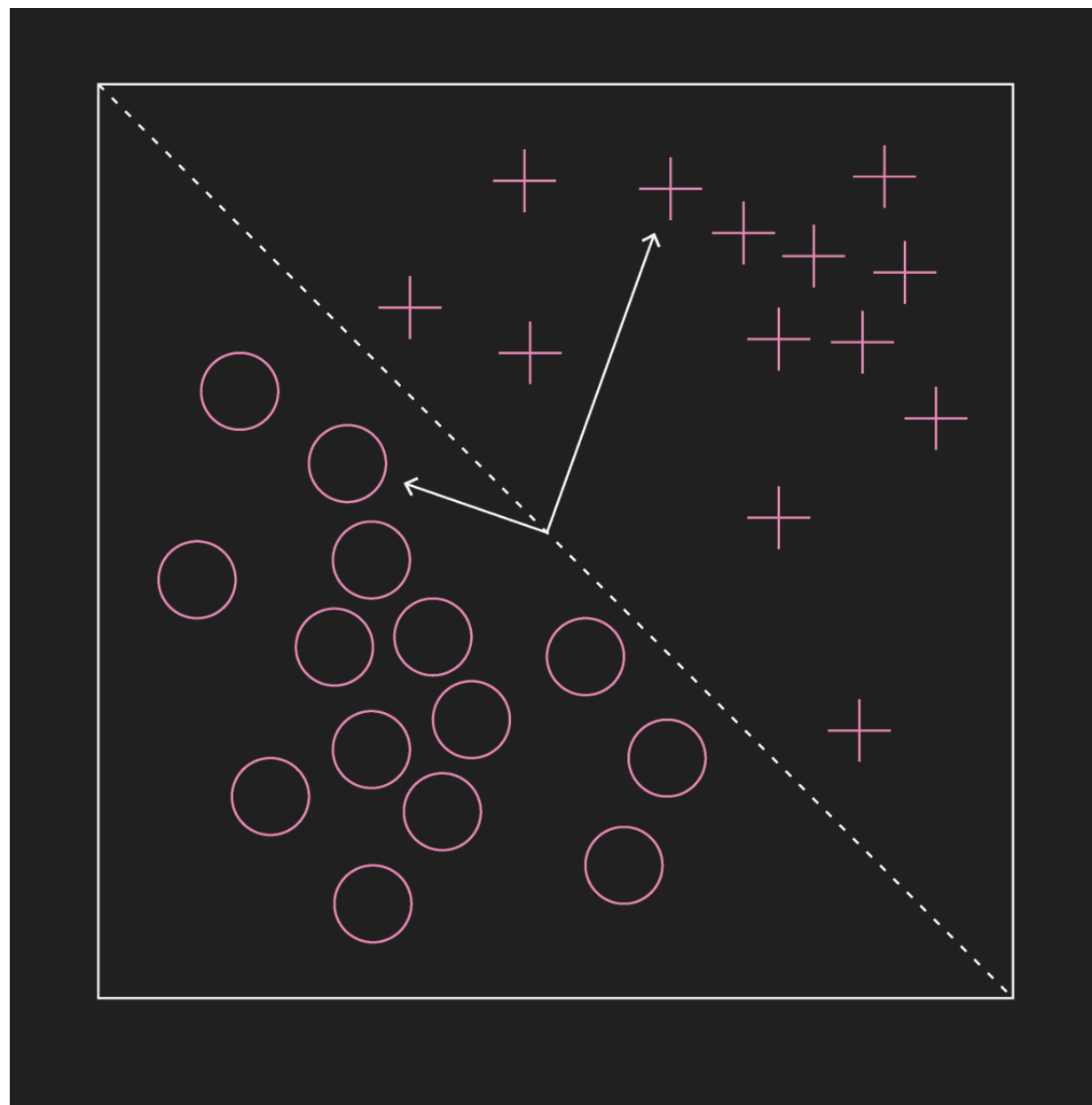
Using Dot-products for measuring similarity!



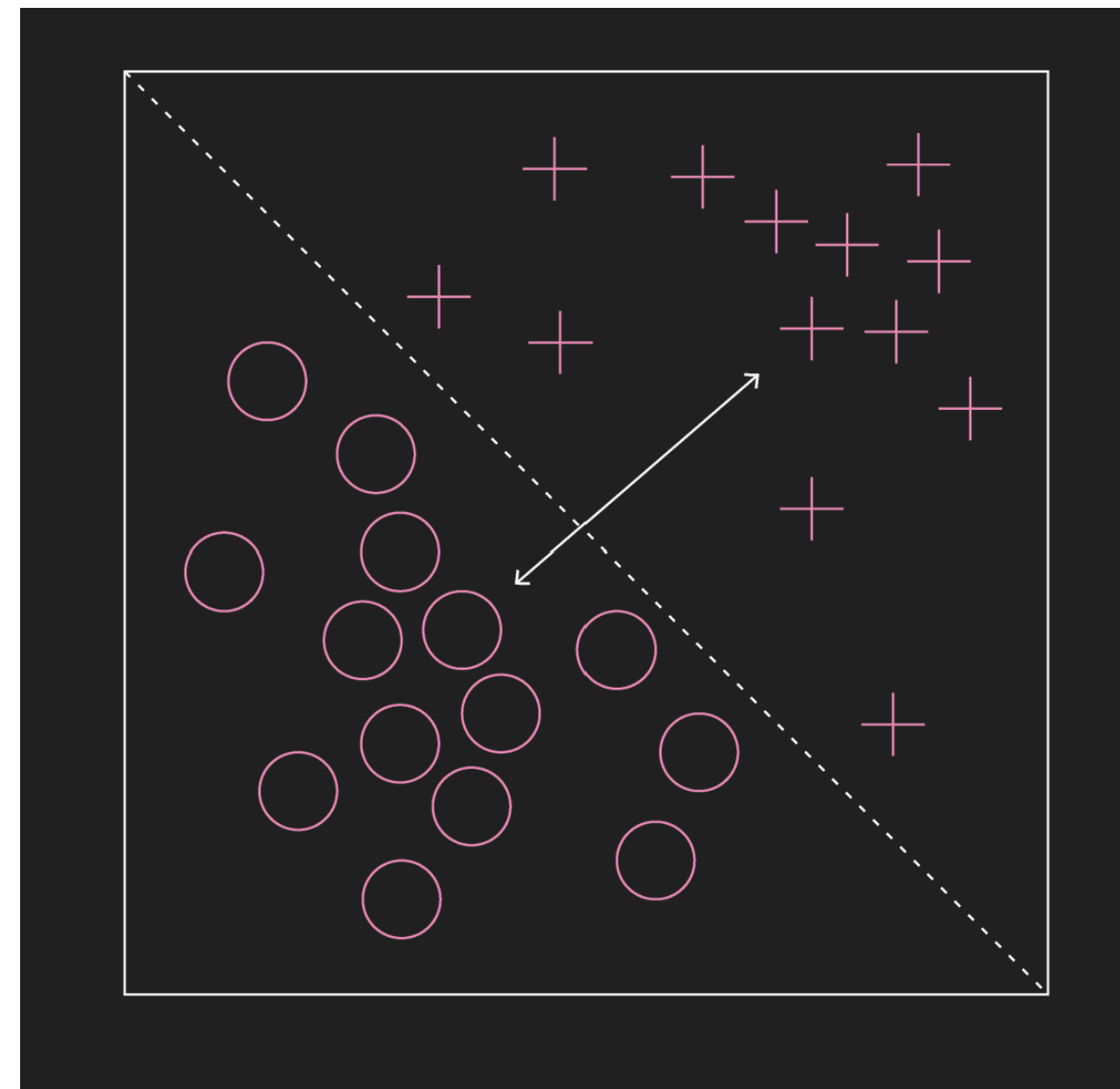
- Multiplying same components of vectors to get a single number.
- Can be used to show if the direction of the vectors align.
- Works for any dimension of data.
- However, **becomes expensive in high dimensions!**

Using Dot-products for measuring similarity!

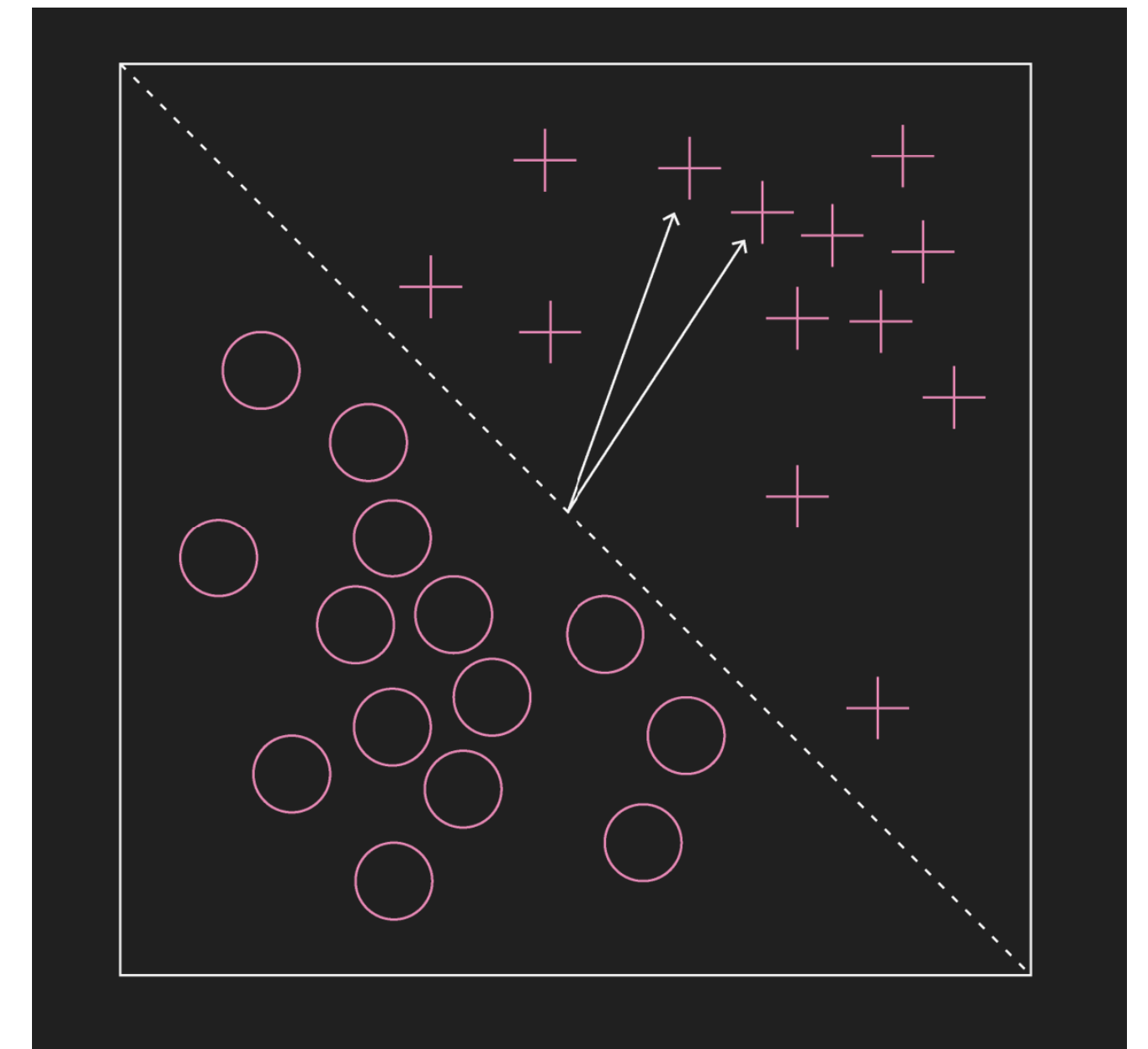
$$\langle \vec{a}, \vec{b} \rangle \text{ or } \vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos(\theta) \\ = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$



Almost right angle: Close to 0

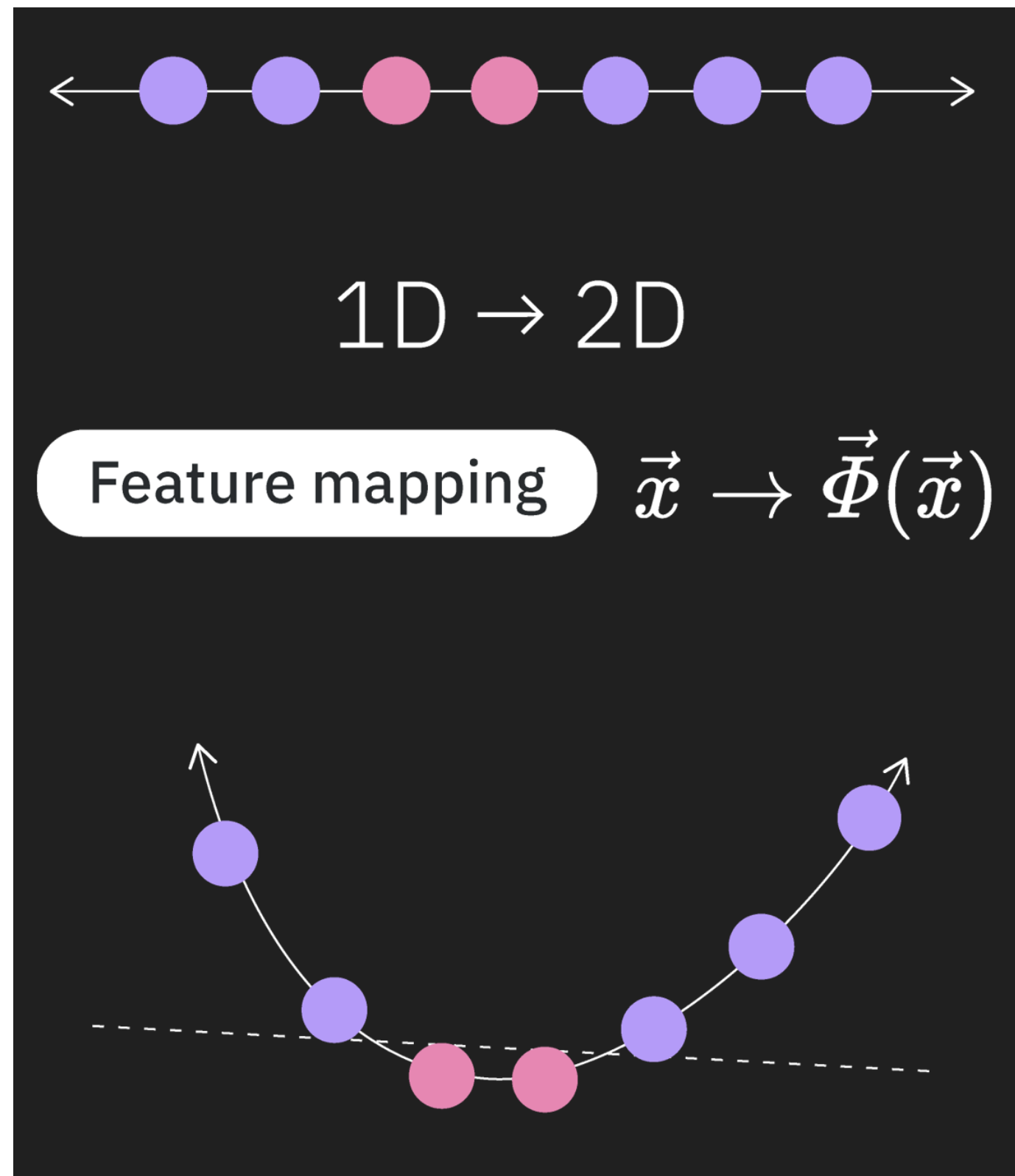


Opposite direction: Negative

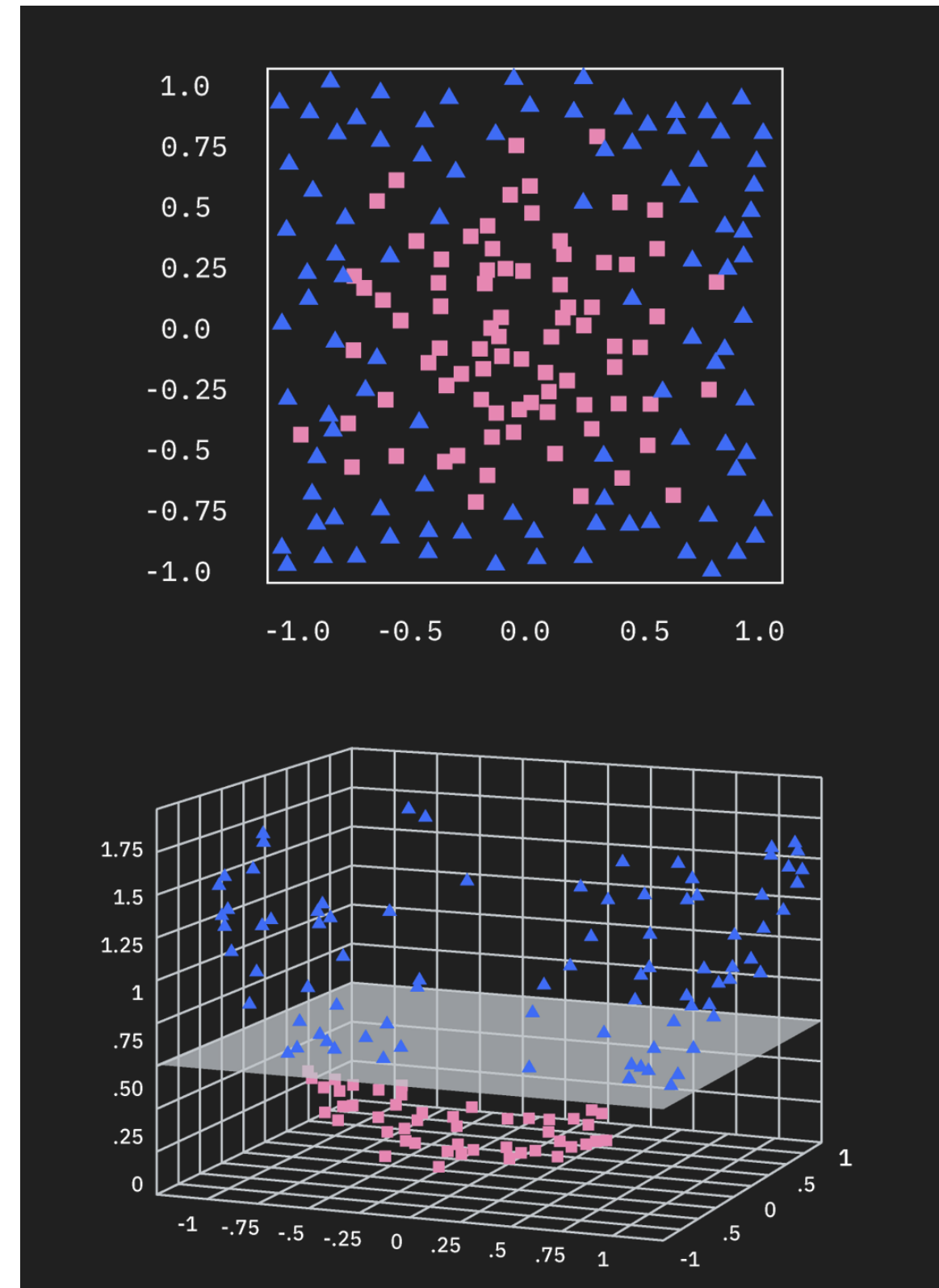


Same direction: Positive

Increasing the dimension increases calculation cost



Example: Mapping 1D to 2D



Example: Mapping 2D to 3D

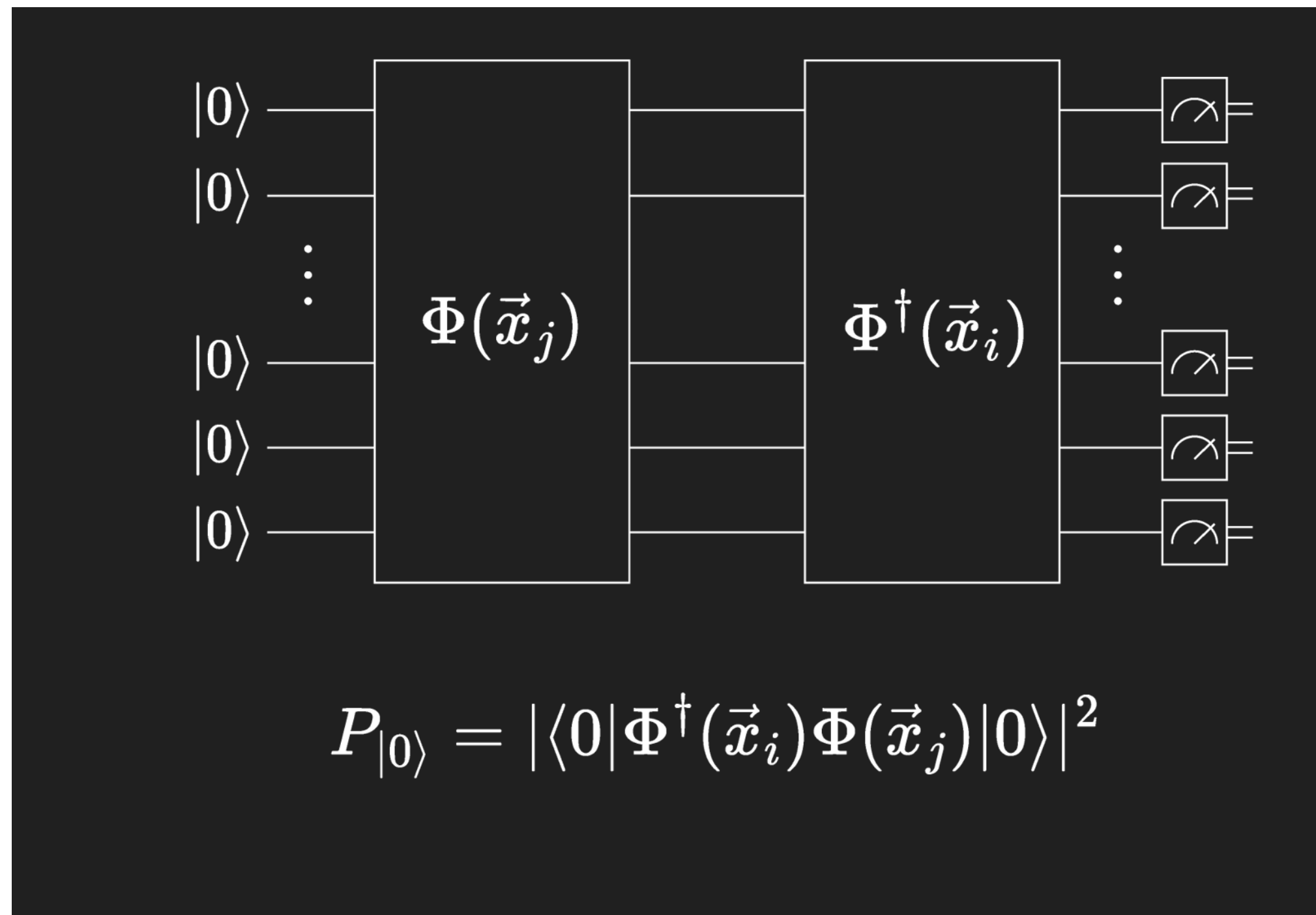
$$\langle \vec{a}, \vec{b} \rangle \text{ or } \vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos(\theta) \\ = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

The Number of operations increases linearly with the dimension. Needing 1 multiplication and 1 addition for each dimension.

What are Kernel Methods?

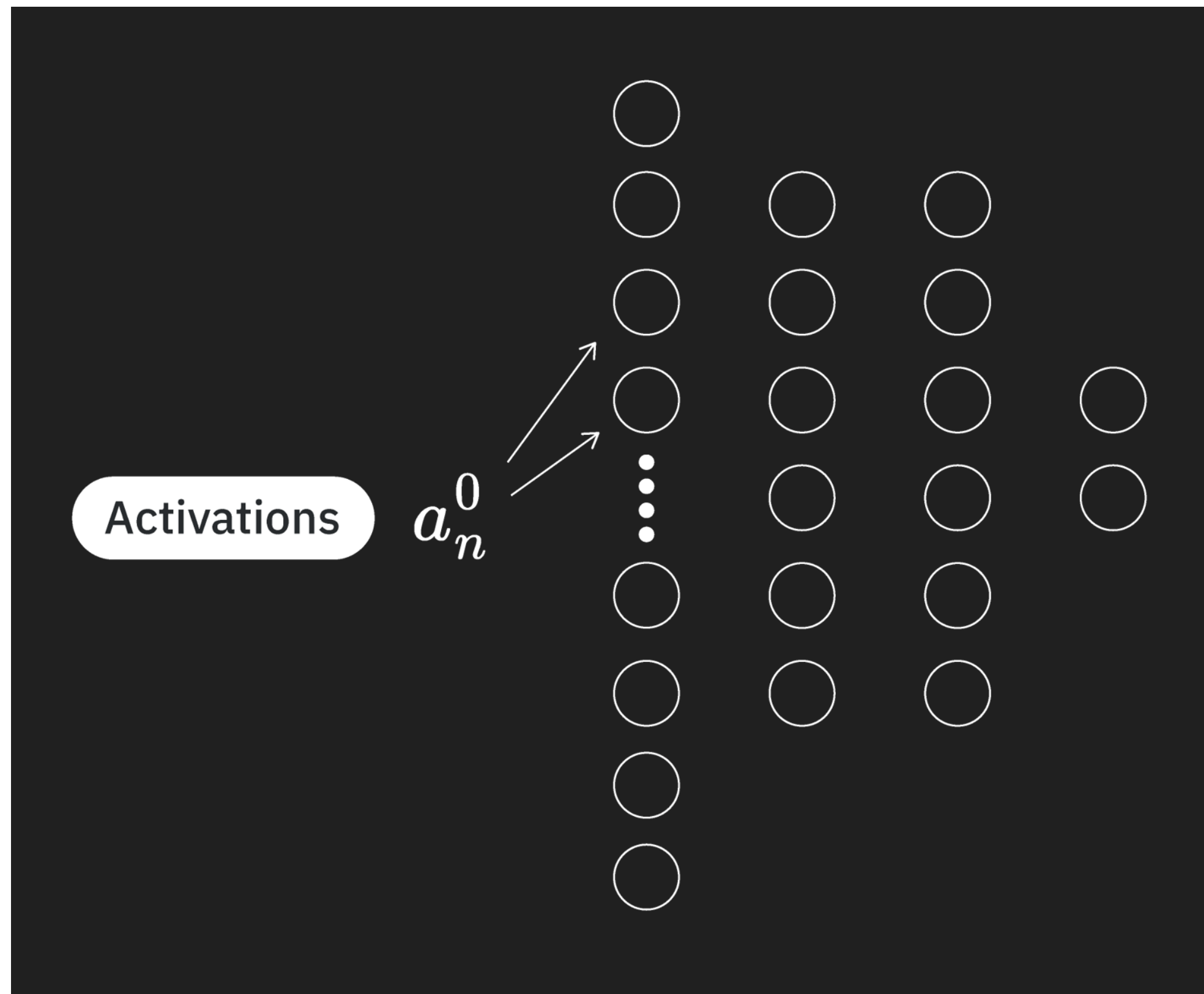
- $\vec{\phi}(\vec{x}) = \vec{\phi}(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$
- $\langle \vec{\phi}(\vec{x}), \vec{\phi}(\vec{y}) \rangle =$
 $= \phi(\vec{x})_1 \phi(\vec{y})_1 + \phi(\vec{x})_2 \phi(\vec{y})_2 + \phi(\vec{x})_3 \phi(\vec{y})_3$
 $= x_1^2 y_1^2 + 2x_1 x_2 y_1 y_2 + x_2^2 y_2^2$
 $= (x_1 y_1 + x_2 y_2)^2$
 $= \langle \vec{x}, \vec{y} \rangle^2$

- Kernel methods are just one of many ways that we can map data into higher dimensions.
- Kernel methods are chosen such that dot products are efficient to calculate in higher dimensions.
- The base idea is to use symmetry and to construct the mapping, such that we can calculate the dot product in the lower dimension.
- In the example on the left a 2 dimensional vector is mapped to 3 dimensions, however, we can express $\langle \vec{\phi}(\vec{x}), \vec{\phi}(\vec{y}) \rangle$ as a function of $\langle \vec{x}, \vec{y} \rangle$.
- Finding such a specific mapping is a non-trivial.



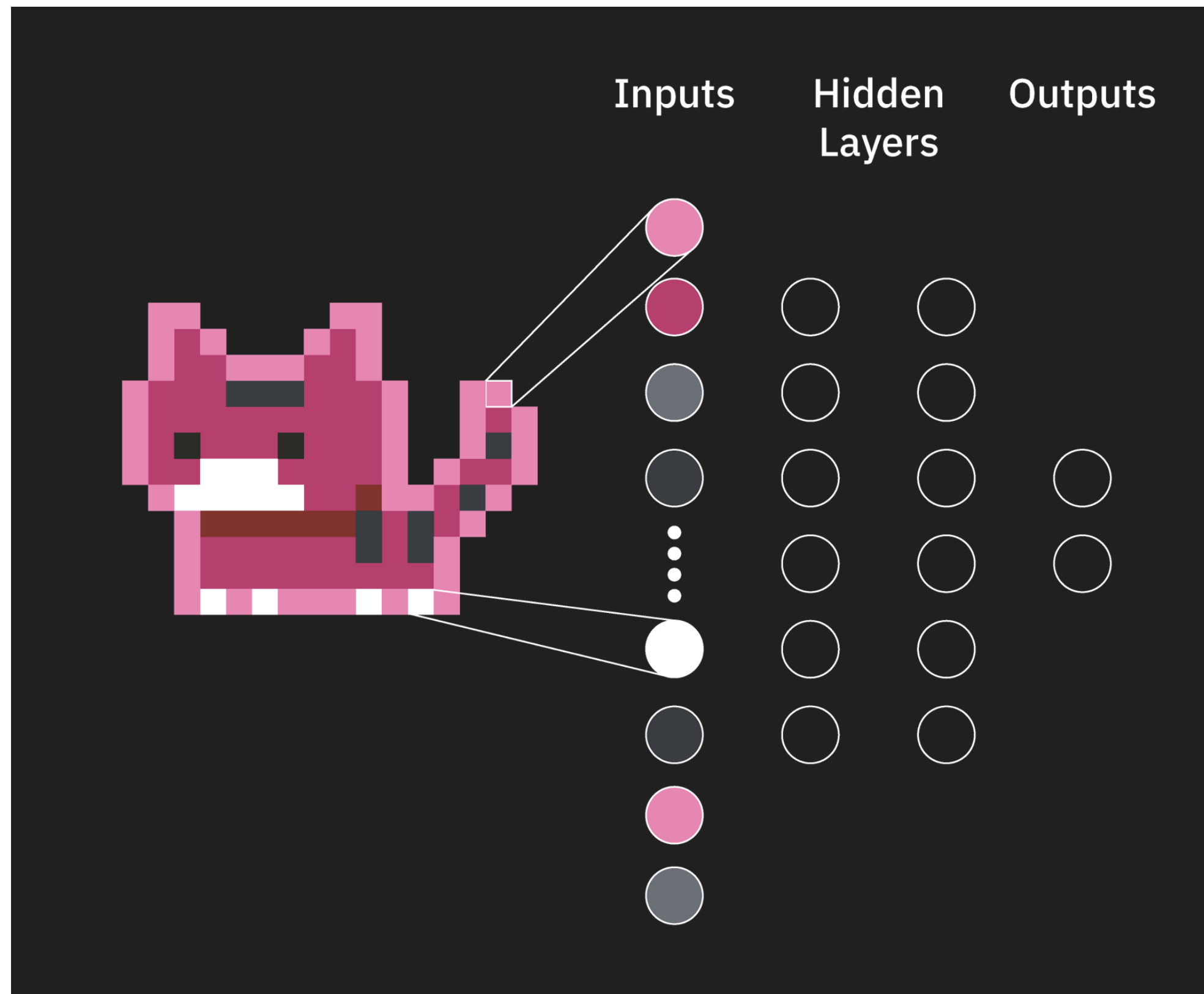
- Idea: we can use a quantum circuit to calculate the dot-product.
- Dot-product comes naturally in quantum computing.
- The probability to measure a specific vector given another vector is exactly the (normalized) dot-product!
- This means we only need to encode the vectors as quantum states.
- This leaves us several different ways to map the data.

Quantum kernels + Support Vector Machine = QSVM (more in the Hands-on)



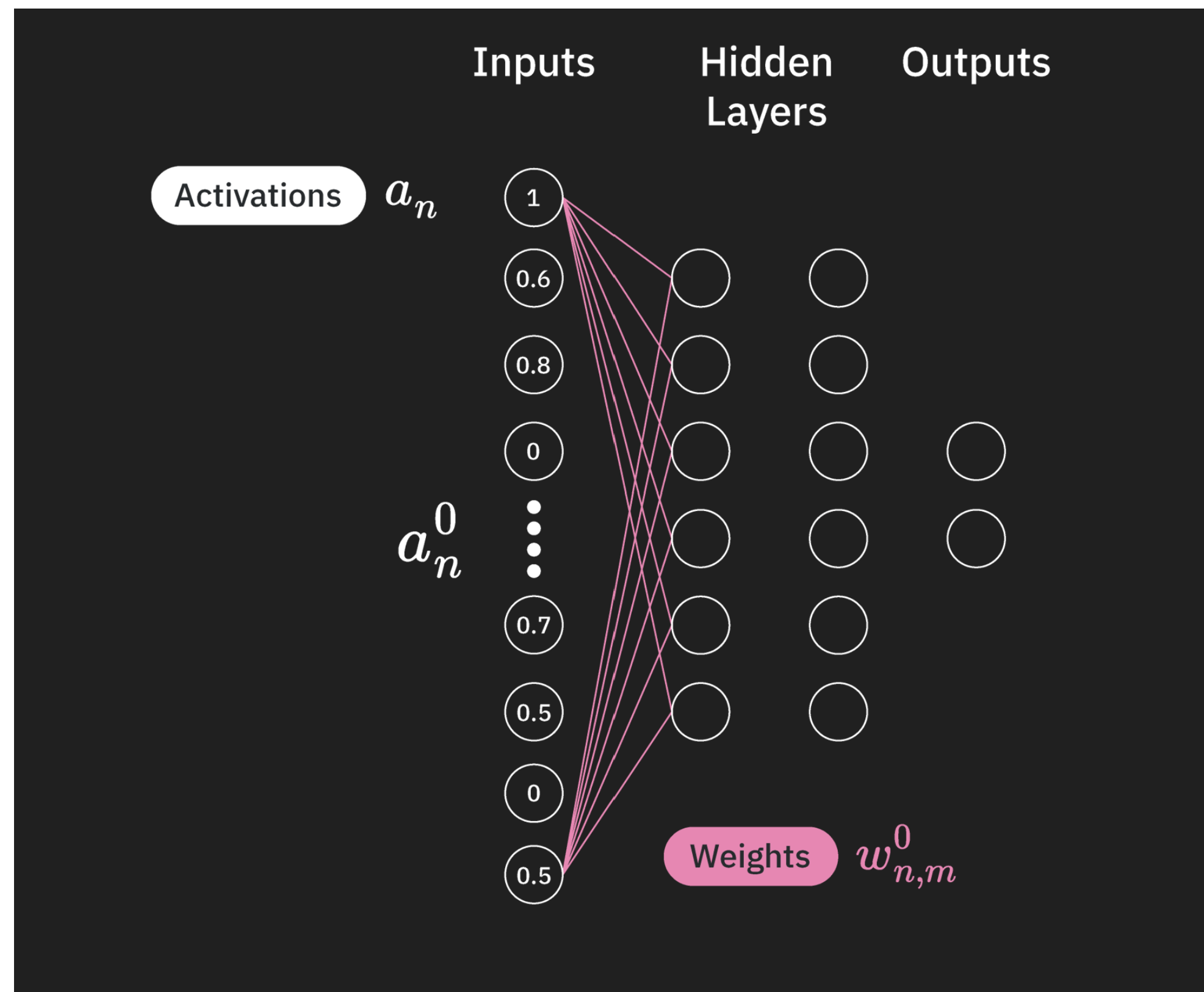
- Quantum Circuit that often leverages structures similar to classical neural networks, often called Quantum Neural Networks.
- We first [start](#) with an [overview of classical neural networks](#).

Classical Neural Network Overview



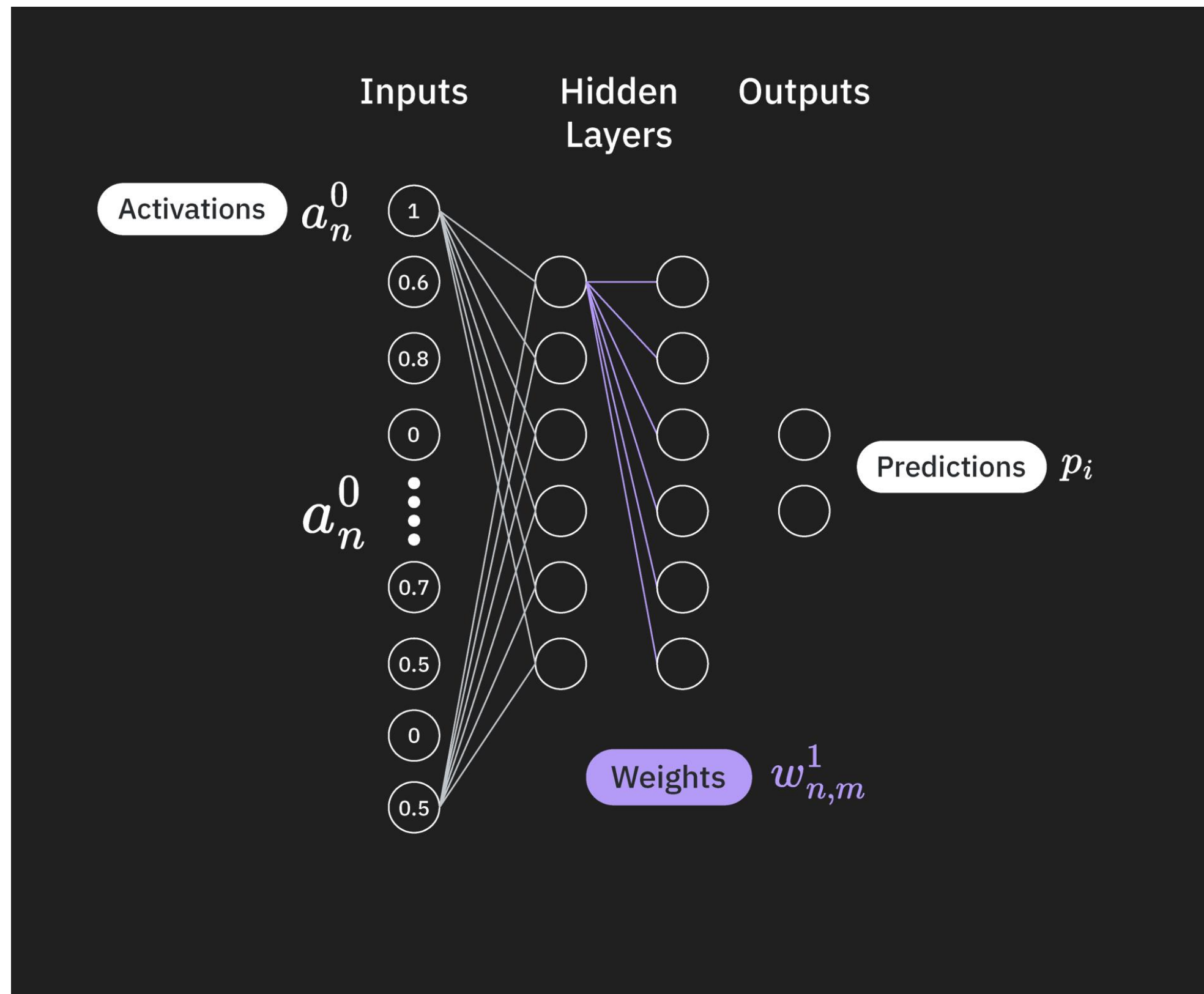
- Data is encoded into the input layers.
- Here, each pixel is stored in a different neuron.

Classical Neural Network Overview



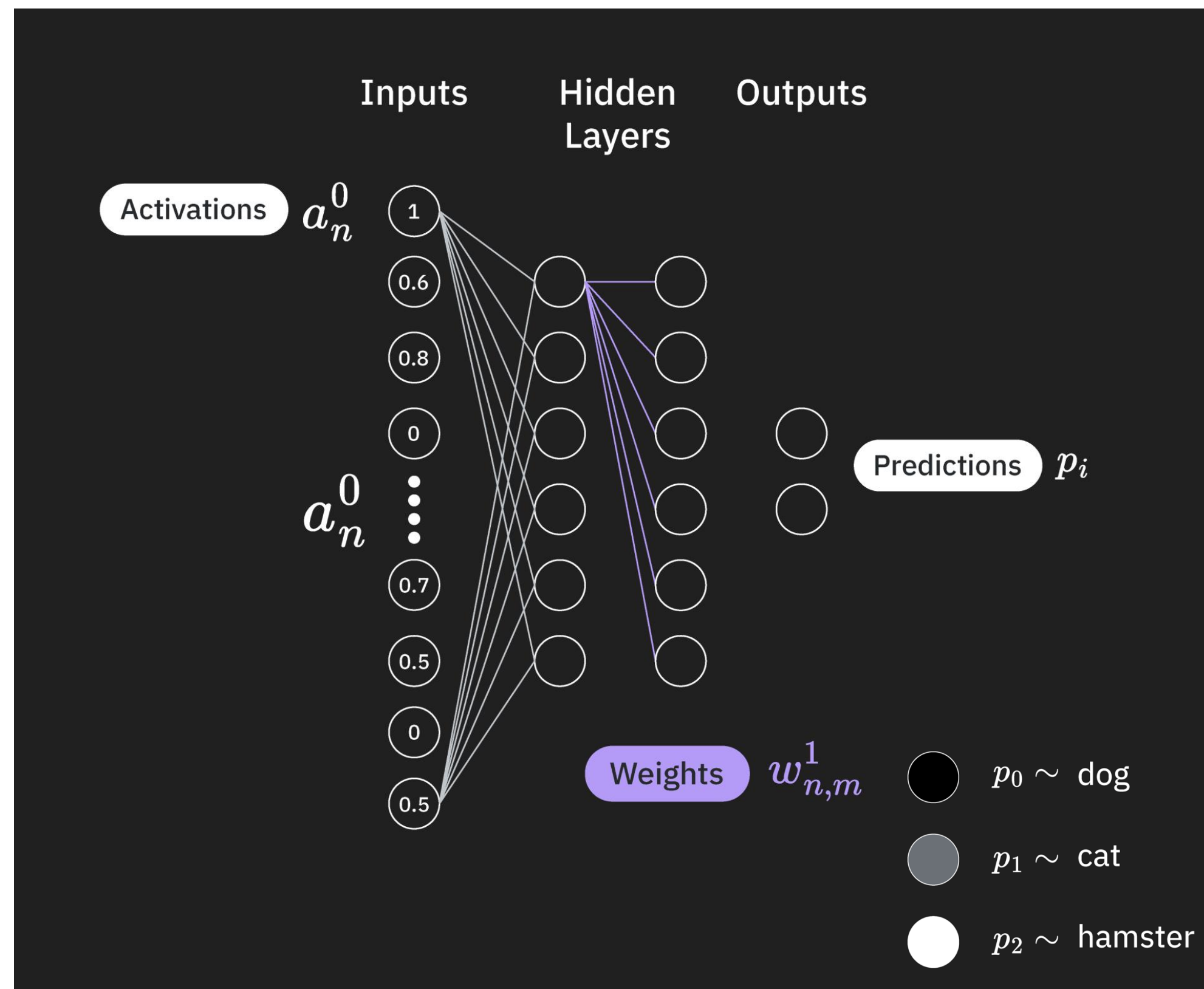
- The input layer is connected with a first internal hidden layer with different weights.
- The Hidden layer can use less neurons than the input layer.
- The weights are the important parts, and represent the model.
- During training the goal is to find the optimal weights.

Classical Neural Network Overview

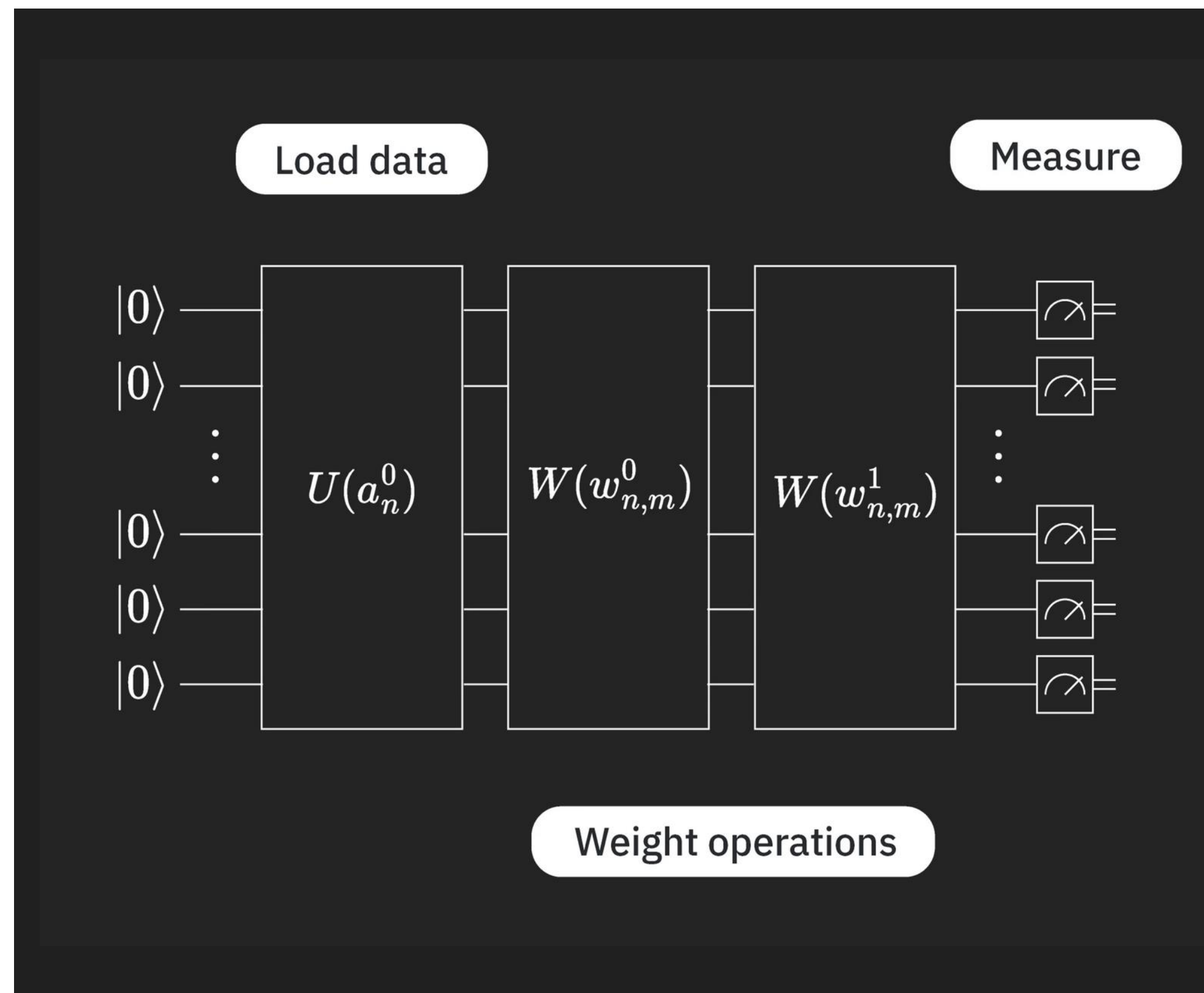


- There can be several hidden layers which are connected to the layer before and to the next layer.
- We again have different weights here which are part of the model and are determined during training.

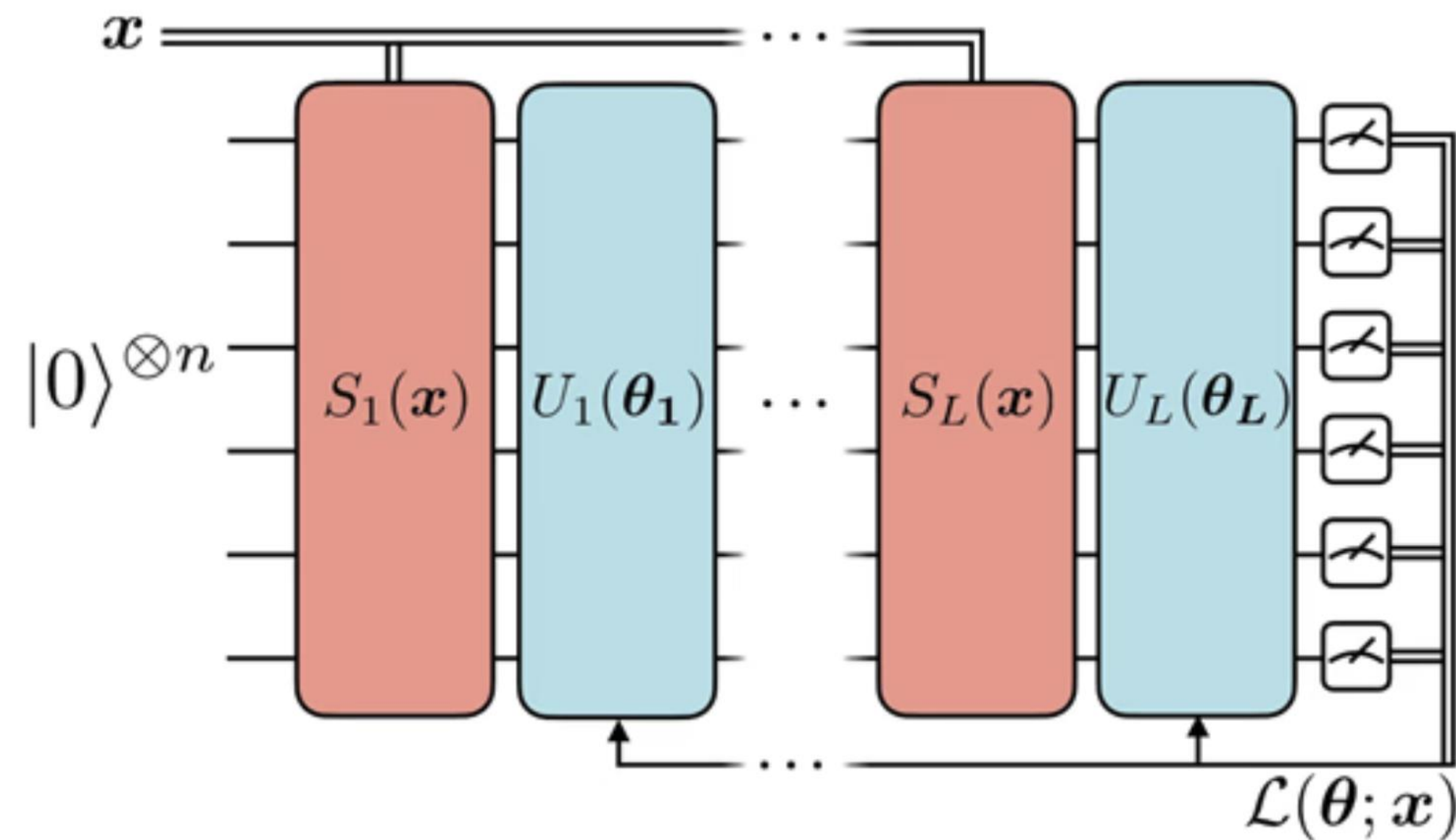
Classical Neural Network Overview



- The last Hidden Layer is connected with the Output Layer, which can be significantly smaller.
- This is the case since we are only interested in classifying data, so the output layer only needs to be able to encode all the different groups.



- The quantum scheme is quite similar.
- The data loading and weightings are linear operations.
- The measurements are non-linear.
- The weight circuits have variational parameters which will be optimized classically
- The biggest difference is that the number of qubits has to stay the same, unlike the neurons in the different levels.
- However, how the weights are applied and how the measurement data is used can be chosen.



- Data reuploading basically means that the input is repeatedly applied after each layer.
- Theoretical results show that this can reduce the number of qubits needed significantly.
- Data reuploading can be used to allow quantum neural networks to approximate complex functions

Pérez-Salinas, Adrián, et al. "Data re-uploading for a universal quantum classifier." *Quantum* 4 (2020): 226.

Overview

- ✓ • **Introduction**
 - What is machine learning?
 - Motivation: Why use Quantum Machine Learning?
- ✓ • **Overview machine learning methods**
 - Types of Classical Machine Learning
 - Going from Classical to Quantum
 - Support vector machines
 - Kernel Methods
 - Other types of Quantum Machine Learning
 - Quantum Neural Networks
- **Data encoding methods**
 - Lossless vs Lossy Encoding
 - Basis Encoding
 - Amplitude Encoding
 - Angle Encoding
 - Phase Encoding
 - Dense Angle Encoding

• Basis Encoding

$$2^4 = 16$$

$$(1, 0, 0, 0), (0, 1, 0, 0), \dots (1, 1, 1, 1)$$

Examples:

single digit $\rightarrow$ in binary $\rightarrow$ encoded in 4 qubits
9 1001 $|1001\rangle$

3d vector $\rightarrow$ in binary $\rightarrow$ encoded in 12 qubits
 $\vec{x} = [3, 9, 1]$ 3 $\rightarrow$ 0011 $|001110010001\rangle$
 9 $\rightarrow$ 1001
 1 $\rightarrow$ 0001

- The simplest form of encoding: qubits are used to directly store bits, so no loss of data.
- It uses a lot of qubits.
- The encoding can be really efficiently calculated.
- Reading the data out can be done efficiently.
- Also used in other algorithms like Shor's algorithm.
- Superposition can be used to encode more than one feature of a data point, similar to amplitude encoding.

• Amplitude Encoding

$$a(1, 0, 0, 0) + b(0, 1, 0, 0) + c(0, 0, 1, 0) + \dots$$

$$a, b, c \dots \in \mathbb{C}, |(a, b, c \dots)|^2 = 1$$

$$\vec{x} = [3, 6, 2]$$

normalise

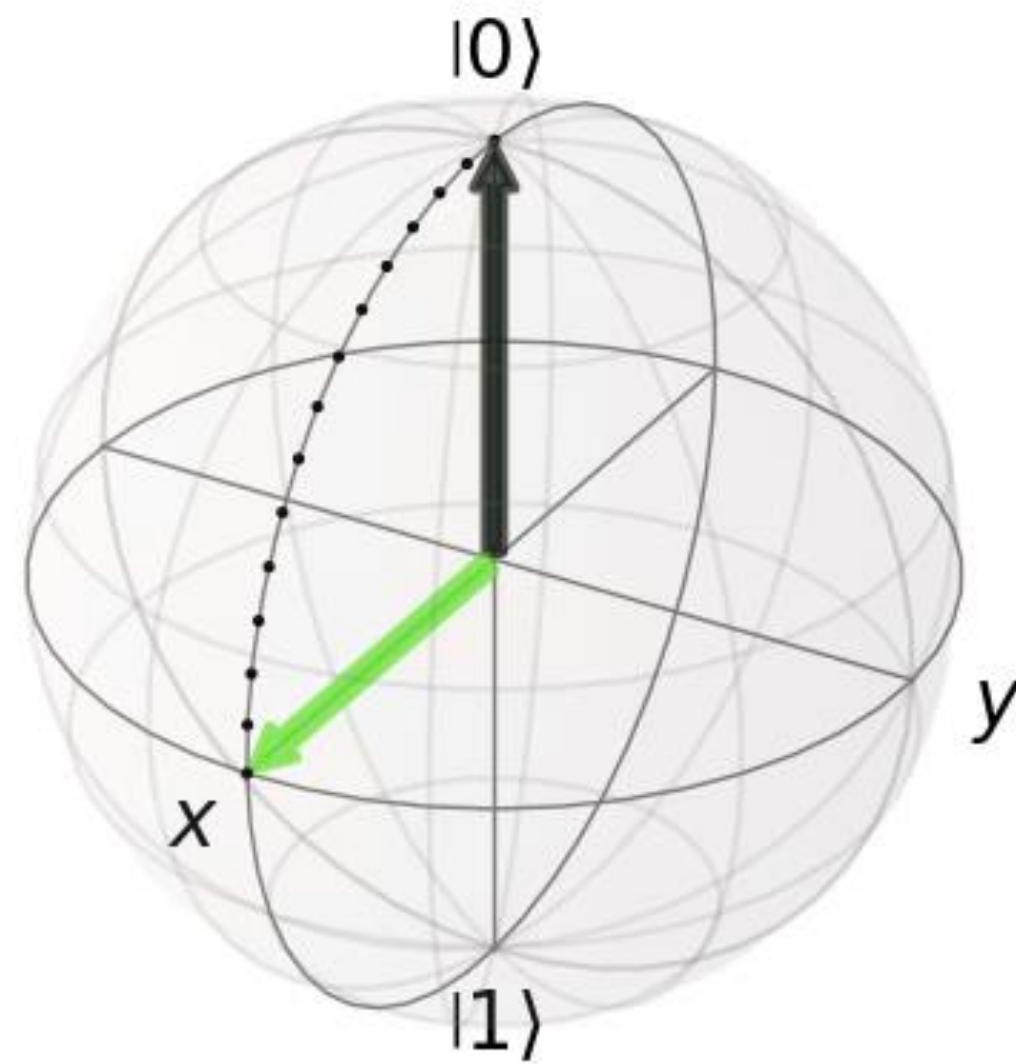
$$3^2 + 6^2 + 2^2 = 49$$

encode

$$|\vec{x}\rangle = \frac{1}{\sqrt{49}} (3|00\rangle + 6|01\rangle + 2|10\rangle + 0|11\rangle)$$

- Data is encoded in the probability amplitudes of a quantum state.
- You can store an exponential amount of data in an n-qubit quantum state.
- Precise encoding of a given quantum state can be really inefficient.
 - Lossy encoding helps by finding a near quantum state which can be more efficiently encoded.
- Readout of the encoded data needs an exponential amount of measurements.
- Example: encoding an image.

• Angle Encoding

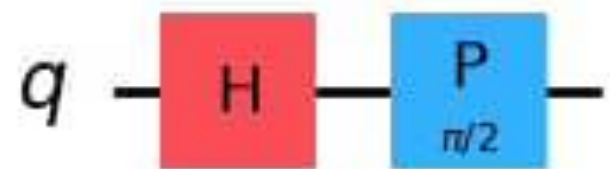
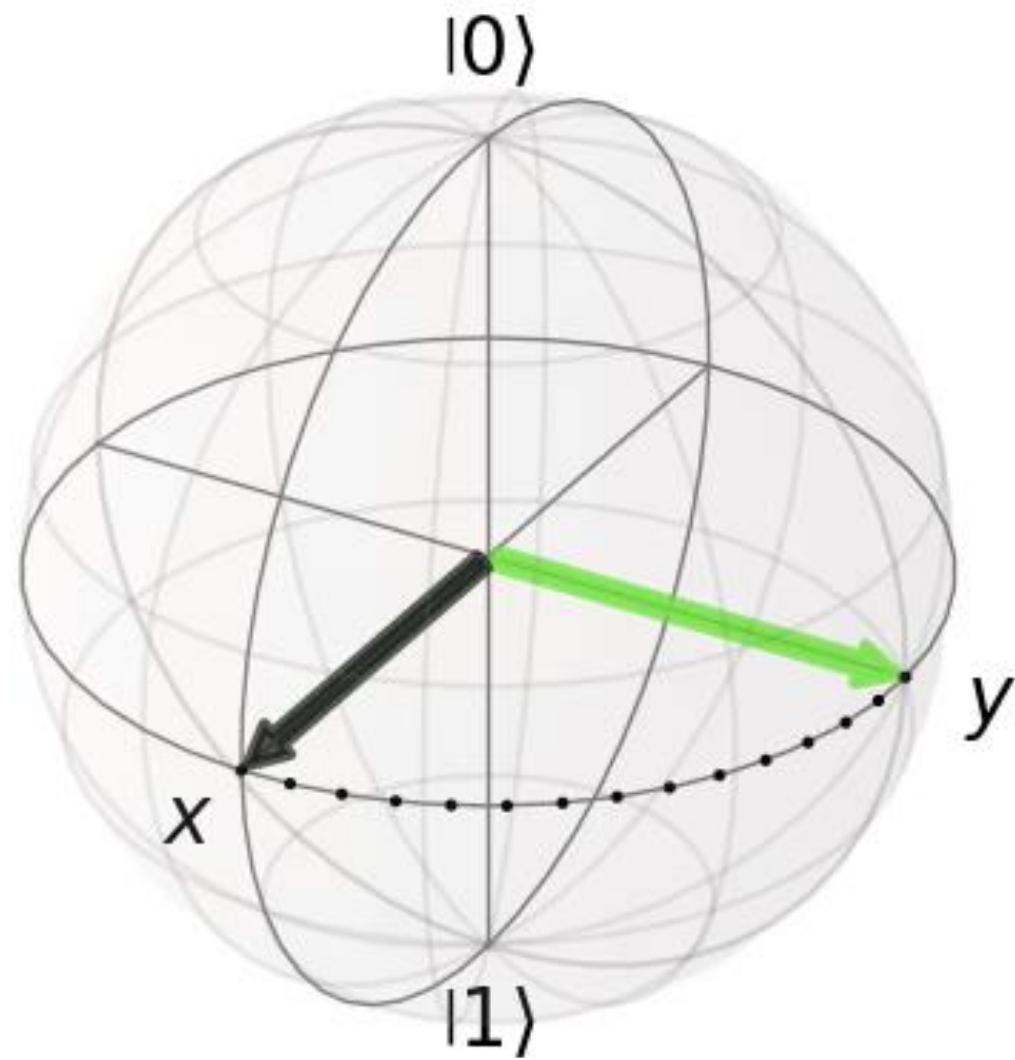


- Data is encoded into single qubits in the [angle of a single rotation](#) of the qubits.
- More efficient than simple basis encoding because an angle can contain more information than a single bit.
- Typically used in QNN

$$\vec{x} = [\pi/2]$$

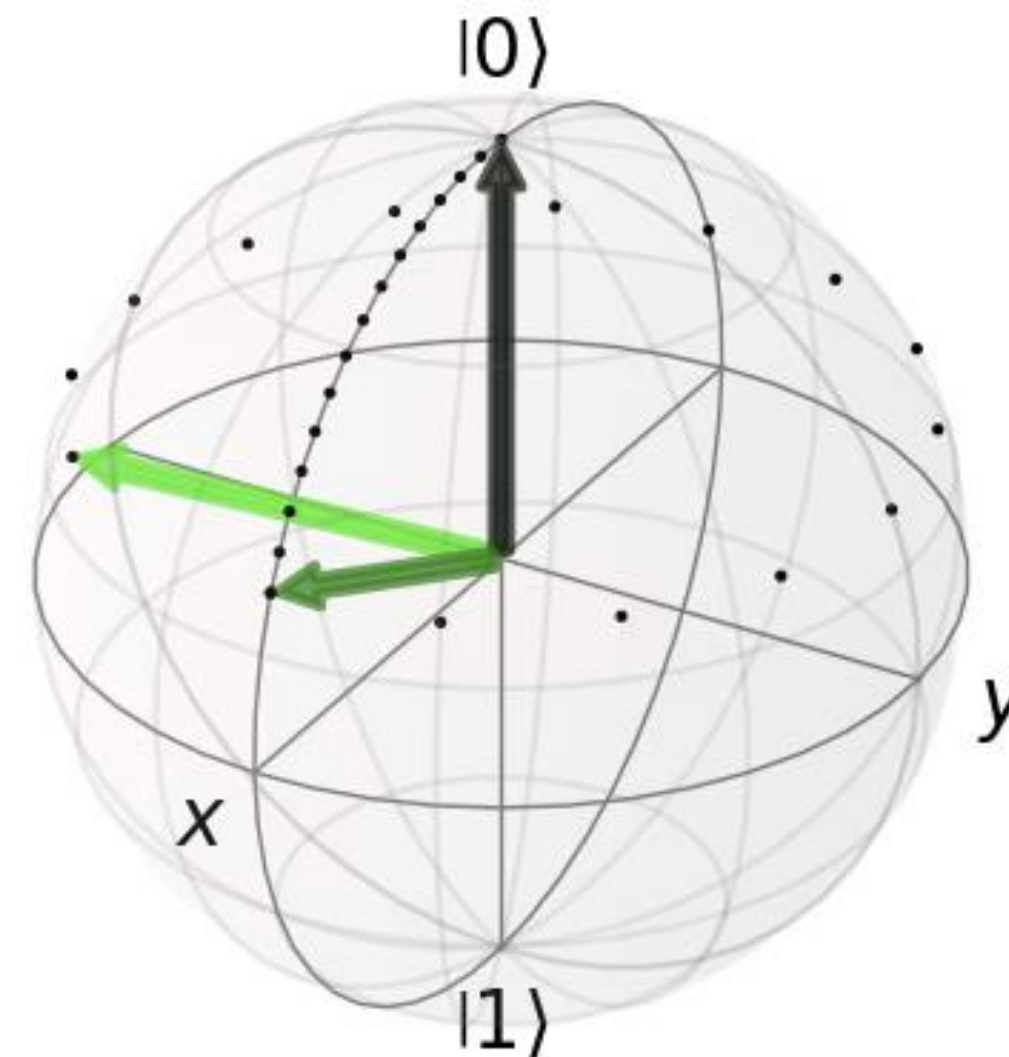
$$|\vec{x}\rangle = R_Y\left(\theta = \frac{\pi}{2}\right) |0\rangle = \cos\left(\frac{\pi}{4}\right) |0\rangle + \sin\left(\frac{\pi}{4}\right) |1\rangle$$

• Phase Encoding and Dense Angle Encoding



$$\vec{x} = [\pi/2]$$

$$\begin{aligned} |\vec{x}\rangle &= P\left(\theta = \frac{\pi}{2}\right) |+\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle + e^{i\pi/2} |1\rangle) \end{aligned}$$



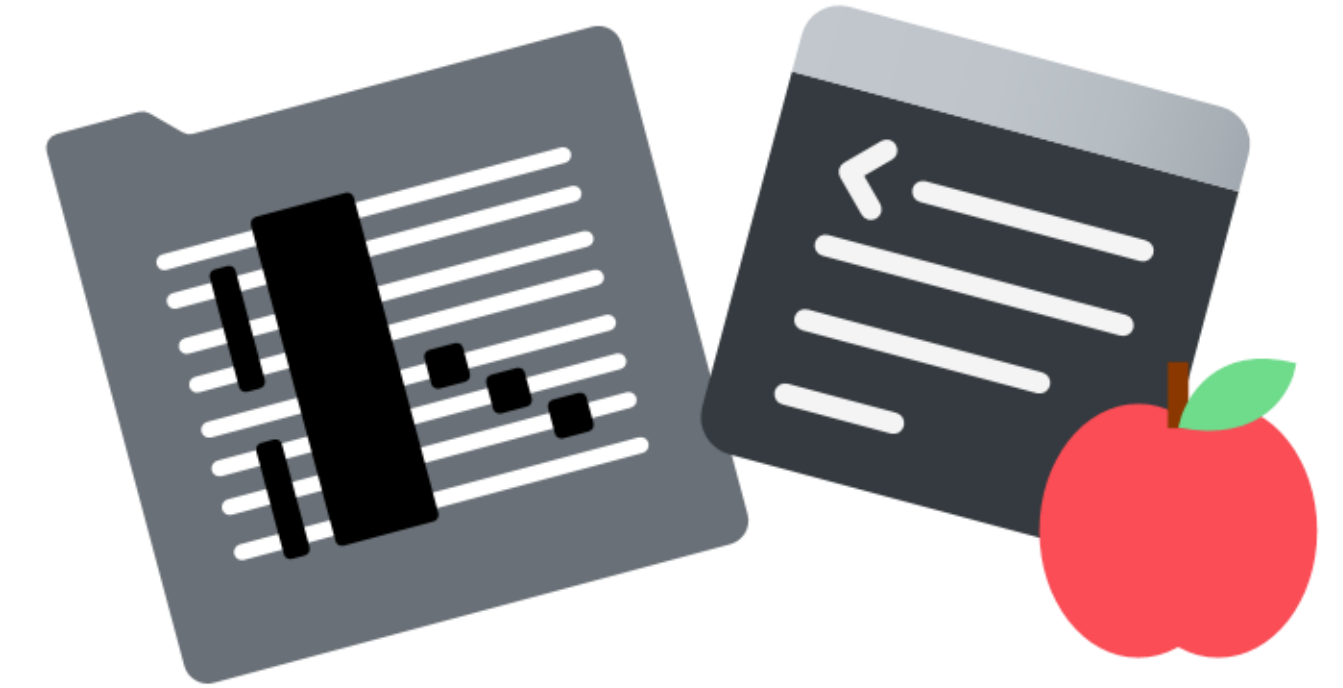
$$\vec{x} = \left[\frac{3\pi}{8}, \frac{7\pi}{4} \right]$$

$$= R_Z\left(\phi = \frac{7\pi}{4}\right) R_Y\left(\theta = \frac{3\pi}{8}\right) |0\rangle$$

- In phase encoding, data is encoded into the **phase** of a qubit using the Phase Gate
- Angle and phase encoding can also be **combined** in the so-called **dense angle encoding**, in which two features are encoded in a single qubit

Recap

- ✓ • **Introduction**
 - What is machine learning?
 - Motivation: Why use Quantum Machine Learning?
- ✓ • **Overview machine learning methods**
 - Types of Classical Machine Learning
 - Types of Quantum Machine Learning
 - Support vector machines
 - Kernel Methods
 - Variational Quantum Classifier
 - Quantum Neural Networks
- ✓ • **Data encoding methods**
 - Lossless vs Lossy encoding
 - Basis Encoding
 - Amplitude Encoding
 - Angle Encoding
 - Phase Encoding
 - Dense Angle Encoding



Further reading

Courses:

Quantum Machine Learning:

<https://quantum.cloud.ibm.com/learning/en/courses/quantum-machine-learning>

Tutorials:

Quantum kernel training:

<https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-kernel-training>

Video:

Quantum Machine Learning:

<https://www.youtube.com/watch?v=ChPLYpUzsY4>